How Our System Works — A Complete Walkthrough

Plain-English and technical, with every term explained. Project: Digital-Twin-Verified Self-Healing of the O-RAN Open-Fronthaul Synchronization Plane.

How to read this: each part is explained first in everyday language, then in technical terms. Boxes marked KEY TERMS define the jargon the first time it appears. Boxes in monospace show what our code actually does. You do not need any prior networking knowledge to follow it.

0. The whole idea in 30 seconds

A modern mobile base station is split into two boxes — a **radio unit** near the antenna and a **processing unit** some distance away — joined by a cable. For the two to work together, their clocks must agree to within about a ten-millionth of a second. If that timing slips, calls drop and the base station can fail; a deliberate *timing attack* can crash it in about two seconds.

Our system continuously watches the timing signal, decides whether a wobble is an **accident (a fault)** or **sabotage (an attack)** — because the two need opposite fixes — then **rehearses the fix inside a digital twin** (a fast software copy) and applies it only if the twin shows it works, all before the base station can fail. Everything below is that idea, built in software, part by part.

> **Analogy.** Think of an orchestra: the two boxes are musicians, and the timing signal is the conductor. A tiny timing error ruins the music. Our system is a smart assistant that spots the wobble, works out whether the conductor is merely tired (fault) or an impostor (attack), tests the correction on a rehearsal recording (the digital twin), and fixes it before the concert collapses.

1. The world our system lives in

Before the parts make sense, here is the setting, with each term defined.

KEY TERMS
O-RAN — “Open Radio Access Network.” A way of building mobile networks from interoperable, multi-vendor parts instead of one sealed box.
O-DU / O-RU — The two split halves of the base station. O-RU (Radio Unit) sits at the antenna; O-DU (Distributed Unit) does the digital processing.
Open Fronthaul — The Ethernet link between O-DU and O-RU (functional “split 7.2x”, carried over eCPRI packets).
S-plane — The “synchronization plane” — the part of the fronthaul that carries timing/clock information. This is the layer our project works on.
PTP (IEEE-1588) — Precision Time Protocol — messages that let one clock (a “slave”) align itself to a master clock over Ethernet, to nanosecond accuracy.
SyncE — Synchronous Ethernet — a hardware method that keeps clock frequency aligned using the Ethernet signal itself.
GNSS — Satellite time (e.g. GPS). Often used as the trusted master time reference. If it is lost, the clock enters “holdover”.
Grandmaster — The reference master clock everyone synchronizes to (often GNSS-disciplined).
Time error — How far the slave clock is from correct time, in nanoseconds (ns). The budget is roughly 100 ns.
Offset / Path delay — Offset = the measured clock difference. Path delay = how long messages take across the link. Both are read from PTP.
PDV — Packet-Delay-Variation — jitter in that path delay, usually caused by network congestion.
Holdover — When the reference is lost, the clock “coasts” on its own internal oscillator and slowly drifts.
LLS-C1..C4 — Four standard ways (O-RAN WG4) to deliver timing to the O-RU — e.g. LLS-C4 uses a local GNSS at the radio. These are our “failover” options.
Spoofing / Replay — Two attacks: spoofing injects fake timing messages (a forged grandmaster); replay re-sends old genuine messages to confuse the clock.

2. The system at a glance — eight parts

The whole project is a pipeline of eight software parts. One command (`run_all.py`) runs them in order, end to end, on a normal laptop with no special hardware.

#	Part	In one line
1	S-plane simulator	Software stand-in for the real timing link; produces realistic timing data.
2	Fault & attack injectors	Deliberately break the timing in known, timestamped ways to auto-label the data.
3	Feature extraction	Turn raw timing streams into short numeric summaries the AI can learn from.
4	Dataset builder	Assemble everything into a labelled dataset + a datasheet.
5	Fault-vs-attack discriminator	The AI that decides: benign fault (H0) or malicious attack (H1)?
6	Digital twin	A fast “what-if” model that predicts how a fix will affect the timing.
7	Governed self-healing loop	The decision-maker: detect → classify → twin-verify → apply the best fix.
8	Benchmark	Grades our loop against simpler strategies and produces the results.

Part 1 — The S-Plane Simulator

File: fronthaul_sim/simulator.py

In plain words

We do not own a real radio unit or timing hardware, so we wrote a small physics model that **behaves like** the timing link. It imitates a slave clock constantly trying to match the master clock, complete with realistic noise and slow drift. This gives us unlimited, repeatable timing data to work with.

The technical terms

KEY TERMS
Clock servo — A feedback controller that nudges the slave clock toward the master each step, proportional to the measured error (like cruise control for time).
Servo gain — How aggressively the servo corrects (our value 0.26). SyncE gain (0.08) similarly corrects frequency.
Drift (ppb) — Parts-per-billion — how fast an uncorrected clock wanders. Our oscillator drifts a few ppb per step.

What the code actually does

Every 20 milliseconds it takes one step: it measures the current offset (with random measurement noise added), applies a correction equal to `servo_gain × offset`, lets SyncE nudge the frequency, and adds a little drift. It records a row of telemetry each step.

```
offset = offset + (freq_error * dt) - servo_gain * measured_offset
freq = freq - synce_gain * freq + small_random_noise
row = {offset, path_delay, pdv, freq_error, synce_ql,
ptp_seq_id, ptp_msg_type, gnss_available, holdover}
```

Result: a healthy run keeps the time error comfortably inside the 100 ns budget — proving the simulated clock is stable before we start breaking it.

Part 2 — Fault & Attack Injectors

File: faults/injectors.py

In plain words

To teach an AI the difference between a fault and an attack, we need examples of both, correctly labelled. So we deliberately inject problems at known times; because we know exactly when each problem starts and ends, every data point is **auto-labelled** as healthy, fault, or attack. This labelled dataset is one of the project's main deliverables — the very thing the research field says is missing.

The two categories and their scenarios

KEY TERMS
H0 — benign fault — GNSS-loss/holdover (reference lost, clock coasts), PDV/congestion (network jitter), SyncE degradation (quality drops).
H1 — malicious attack — PTP spoofing (forged grandmaster / fake offset step), replay (re-sent old messages, out-of-order sequence numbers).

Why it is realistic (the "hardening" we did)

A first version was too easy — the AI scored a perfect 100%, which looks fake. We fixed that so the score is **earned**:

- Removed giveaway labels: a real spoof looks like normal traffic, so we no longer stamp attacks with a special message type.
- Overlapping magnitudes: attack and fault sizes now overlap, so size alone cannot separate them.
- Evasive mimicry: ~60% of attacks forge a fault's fingerprint (fake SyncE/GNSS/holdover) to hide.
- Attacks during congestion: many attacks add network jitter so that clue is no longer fault-only.
- Sensor noise: telemetry is read imperfectly, blurring the signals — as in a real network.

Why this matters for your viva. These choices are why the final accuracy is 98.7% with real errors, not a suspicious 100%. The errors concentrate on the hard, evasive attacks — which is exactly the interesting case.

Part 3 — Feature Extraction

File: telemetry/features.py

In plain words

An AI cannot learn from a raw, endless stream of numbers. So we chop the stream into short overlapping windows and, for each window, compute a handful of **summary numbers** (features) that capture its shape.

The technical terms

KEY TERMS
Sliding window — A short time slice (0.4 s, stepped every 0.2 s) over which we summarize the data.
Feature vector — The fixed list of numbers describing one window — the input the AI sees.

The 10 features we compute per window

- offset mean / std / absolute-max — how far off, how jumpy, and the worst moment of the clock error.
- path-delay mean and PDV std — the link delay and its jitter.
- sequence regressions — how often PTP message numbers went backwards (a replay hint).
- message irregularity — fraction of malformed/odd messages.
- SyncE quality max, GNSS-loss rate, holdover rate — categorical health flags.

Each window is labelled by the majority label of the raw rows inside it (healthy / H0 / H1).

Part 4 — Dataset Builder

File: dataset/build.py

This part simply orchestrates Parts 1–3: it runs many scenarios through the simulator and injectors, extracts window features, and saves three things — `splane_telemetry.csv` (raw), `splane_windows.csv` (features), and a **DATASHEET.md** describing the data. The current dataset has **2,508 windows** (1,614 healthy, 536 fault, 358 attack).

KEY TERMS
Datasheet — A short document describing a dataset — how it was made, its labels, and its limitations — so others can reuse it responsibly.

Part 5 — The Fault-vs-Attack Discriminator

File: discriminator/model.py

In plain words

This is the AI that answers the core question: **is this anomaly a fault or an attack?** It learns from the labelled windows and is then tested on windows it has never seen.

The technical terms

KEY TERMS
Random Forest — A machine-learning model made of many small decision trees that vote; robust and good with mixed numeric features. Ours uses 90 trees, depth 6.
Train/test split — We train on part of the data (65%) and test on the rest (35%), so the score reflects unseen data. “Stratified” means the split keeps the class balance.
Class weight = balanced — Tells the model to treat the rarer class as importantly as the common one.
Accuracy / Precision / Recall / F1 — Accuracy = fraction correct. Precision = of those it called ‘attack’, how many were. Recall = of real attacks, how many it caught. F1 = their balance.
ROC-AUC — A 0–1 score of how well the model separates the two classes across all thresholds; 1.0 is perfect, 0.5 is guessing.
Confusion matrix — A 2×2 table of correct vs mistaken predictions.

What the code does, and the result

It trains the Random Forest on the 10 features of the anomalous windows and evaluates it on the held-out set. It also runs a **detection-only baseline** — a stand-in for prior work that simply calls every anomaly an ‘attack’ (an unsafe response). Our discriminator scores **98.7% accuracy, ROC-AUC 0.999**, with only 4 mistakes out of 313; the detection-only baseline scores **40%**. The margin is the value our fault-vs-attack step adds.

Part 6 — The Digital Twin

File: twin/model.py

In plain words

Before the system applies a fix to the (simulated) network, it first tries that fix in a **digital twin** — a fast, lightweight copy that predicts how the timing error would evolve under each candidate action. This is what makes the twin "load-bearing": it is used to decide, not just to make data.

The technical terms

KEY TERMS
Digital twin — A live software model of a real system, used here to answer "what would happen if I did X?" without touching the real thing.
Counterfactual forecast — A prediction of an outcome under an action that has not been taken yet — the twin's core job.
Fidelity score — A 0–1 measure of how much to trust the twin right now; it drops when telemetry is messy (high jitter, malformed messages, sequence regressions).

What the code does

Each recovery action has a modelled effect — a `(decay, floor)` pair describing how quickly it pulls the time error down and where it settles. Given the current worst-case offset, the twin rolls it forward for each action and returns the predicted peak and steady error, plus a fidelity score. Crucially, when fidelity is low, the loop is told *not* to trust the twin and to fall back to the safe default.

```
for each action: value = |value * decay| + floor # roll forward
fidelity = 1 - penalty(pdv, malformed_msgs, seq_regressions)
return peak_error, steady_error, fidelity
```

Part 7 — The Governed Self-Healing Loop

File: healing/loop.py

In plain words

This is the brain that ties everything together and makes the decision — the "response step" that prior work left undefined.

The five steps it runs

- **Detect** — is the window abnormal? (worst offset or jitter above a threshold.)
- **Classify** — ask the discriminator: H0 fault or H1 attack?
- **Shortlist** — pick the sensible recovery actions for that class (attacks → isolate rogue master / GNSS failover; faults → failover / reroute / holdover).
- **Twin-verify** — forecast each candidate in the digital twin; keep the one that best holds timing in budget.
- **Commit or fall back** — apply the best action only if it beats the safe default AND twin fidelity is high enough; otherwise use the conservative safe default. It also records an auditable reason and checks it decided inside the time budget.

KEY TERMS

Action space — The menu of fixes: isolate_rogue_master, failover_gnss, failover_lls_c1/c2/c3, holdover, reroute_path, safe_default.

Safe default — The conservative do-least-harm action the chosen fix must beat before it is applied.

Decision budget — A hard time limit so the whole decision finishes well inside the ~2 s failure window.

Part 8 — The Benchmark

File: benchmark/run.py

In plain words

This is the exam. It runs our loop and several simpler strategies over the same faults and attacks, then scores them so we can prove the loop actually helps.

The strategies compared (baselines)

KEY TERMS
Detection-only / no-response — Sees the anomaly but does nothing — the prior-work stand-in.
Always-holdover — Always coasts on the internal clock, whatever the cause.
Always-failover — Always switches timing source, whatever the cause.

The metrics (what we measure)

KEY TERMS
Recovery success rate — Fraction of cases where timing was restored correctly.
Wrong-action rate — Fraction where a harmful/incorrect fix was chosen.
MTTR — Mean Time To Recover — average time to get timing healthy again.
Peak time error (ns) — The worst clock error reached; must stay under the 100 ns budget.
Within-2 s rate — Fraction that recovered before the failure window closed.

The result

Method	Recovery	Wrong-action	MTTR (s)	Peak err (ns)	Before 2 s
Governed loop (ours)	**0.95**	**0.05**	**0.55**	**86 (in budget)**	**Yes**
Detection-only	0.00	0.00	2.0	224	No
Always-holdover	0.60	0.40	1.18	176	Yes
Always-failover	0.60	0.00	0.62	176	Yes

Read it as: detection alone recovers nothing; the twin-verified loop recovers 95% of cases, keeps the error inside the 100 ns budget, and does so before the crash — with the lowest wrong-action rate.

Part 9 — Reproducibility and Tests

Files: scripts/run_all.py, tests/

A single command, `python scripts/run_all.py`, regenerates the dataset, trains the discriminator, builds the twin forecasts, runs the benchmark, and writes all results and plots. Automated tests (`pytest`, 3/3 passing) check that the healthy clock stays in budget, the dataset labels are valid, and the governed loop restores timing. This means anyone can reproduce every number from scratch.

How to read the headline numbers (plain English)

- **98.7% discriminator accuracy** — out of 313 unseen anomalies, it correctly told fault from attack in all but 4.
- **ROC-AUC 0.999** — it almost always ranks a true attack as more attack-like than a true fault.

- **0.95 recovery / 0.05 wrong-action** — the loop fixed 95% of cases and rarely chose a harmful action.
- **86 ns peak error** — even at its worst, the loop kept the clock inside the ~100 ns safety budget; every baseline blew past it.
- **MTTR 0.55 s** — on average it recovered in about half a second, far inside the 2-second failure window.

Honest limitations (say these openly)

- This is an **emulation**: the clock is a physics-based software model, not hardware-timestamped PTP on real NICs.
- The digital twin is a **simplified** dynamical model, not a full physical-layer simulator.
- The data is **synthetic** (though realistically hardened); it is for research/demo evidence, not certification.
- Next software step (no hardware needed): re-run over the real `linuxptp` daemon with `tc netem` for higher realism.

Bottom line. Every part of the method — simulate, break, label, discriminate, twin-verify, heal, benchmark — is built, runs end-to-end on a laptop, and is reproducible. What remains is realism upgrades (linuxptp/netem, then optional hardware), not missing method.